



Office de la Formation Professionnelle et de la Promotion du Travail
Institut Spécialisé de Technologie Appliquée — ISTA Hay Salam

2eme Année — Développement Digital Web Full Stack

RAPPORT DE PROJET DE FIN D'ETUDES

PharmaCrea

Application Web de Gestion de Pharmacie
Laravel 13 & React 19

REALISE PAR

Milad Dlou & Mouhcine Ait Yahya

ENCADRANT

Pr. Khalid Mzibra

Année Académique 2025 / 2026 | Mai - Juin 2026

Remerciements

Nous tenons à adresser nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Nous remercions en premier lieu notre encadrant Pr. Khalid Mzibra pour ses précieux conseils, son suivi rigoureux et sa disponibilité tout au long de ce projet.

Nous exprimons également notre gratitude envers :

- ▶ L'ensemble du corps professoral de l'ISTA Hay Salam pour la qualité de la formation dispensée.
- ▶ L'Office de la Formation Professionnelle et de la Promotion du Travail (OFPPT) pour le cadre pédagogique structurant.
- ▶ Nos familles et nos proches pour leur soutien moral et leur encouragement continu.

Ce projet représente l'aboutissement de deux années de formation intensive en Développement Digital Web Full Stack. Il nous a permis de mobiliser l'ensemble des compétences acquises — conception, architecture, développement backend et frontend — pour livrer une solution web complète et fonctionnelle.

Resume

Resume (Francais)

Ce rapport presente la conception et la realisation de PharmaCie, une application web full stack dediee a la gestion d'une pharmacie. Le systeme offre aux clients la possibilite de consulter un catalogue de medicaments, de gerer un panier d'achat et de passer des commandes en ligne, tandis que l'administrateur dispose d'un tableau de bord complet pour piloter les produits, les categories et les commandes.

L'architecture repose sur Laravel 13 (backend API REST avec Sanctum) et React 19 (frontend SPA avec Tailwind CSS et React Context), deployes de facon decouplee.

Abstract (English)

This report presents the design and implementation of PharmaCie, a full-stack web application for pharmacy management. The system allows customers to browse a medicine catalogue, manage a shopping cart, and place online orders, while administrators have a complete dashboard to manage products, categories, and orders.

The architecture is built on Laravel 13 (REST API backend with Sanctum) and React 19 (SPA frontend using Tailwind CSS and React Context API), deployed as independent services communicating through secured JSON endpoints.

Mots-cles :

Laravel 13

React 19

API REST

MySQL

Sanctum

Tailwind CSS

Full

Table des Matieres

Remerciements	2
Resume	3
Introduction Generale	5
Chapitre 1 — Presentation du Projet	6
1.1 Contexte et problematique	6
1.2 Objectifs du projet	6
1.3 Parties prenantes	7
1.4 Planning et organisation	7
Chapitre 2 — Analyse et Conception	8
2.1 Acteurs et cas d'utilisation	8
2.2 Diagramme de classe	9
2.3 Modele de la base de donnees	9
2.4 Architecture du systeme	10
Chapitre 3 — Specifications Techniques	11
3.1 Stack technologique	11
3.2 API REST — Endpoints	12
3.3 Securite et authentication	13
Chapitre 4 — Realisation et Tests	14
4.1 Interfaces utilisateur — Cote Client	14
4.2 Interfaces utilisateur — Cote Admin	15
4.3 Tests fonctionnels	16
Conclusion Generale	17
Bibliographie et Webographie	18

Introduction Generale

La transformation numerique touche aujourd'hui tous les secteurs d'activite, y compris celui de la sante. Les officines pharmaceutiques, longtemps gerees a travers des outils traditionnels ou des logiciels non dedies, ressentent de plus en plus le besoin de solutions web modernes capables d'automatiser et de centraliser leurs operations quotidiennes.

Dans ce contexte, nous avons concu et developpe PharmaCie, une application web full stack dediee a la gestion d'une pharmacie. Ce projet de fin d'etudes, realise dans le cadre de notre formation en Developpement Digital Web Full Stack a l'ISTA Hay Salam, ambitionne de repondre a deux problematiques complementaires :

- Cote pharmacien (administrateur) : disposer d'un outil de gestion centralise pour les produits, les stocks, les categories et les commandes clients.
- Cote client : beneficier d'une vitrine numerique permettant de consulter le catalogue, de gerer un panier et de passer des commandes directement en ligne.

L'application est construite sur deux piliers technologiques modernes et complementaires : Laravel 13 pour le backend (API REST securisee) et React 19 pour le frontend (application monopage reactive). Ces choix techniques refletent les exigences actuelles du marche et les competences enseignees au sein de notre formation.

Structure du rapport

Ce rapport est organise en quatre chapitres : le Chapitre 1 presente le contexte et les objectifs ; le Chapitre 2 detaille la phase d'analyse et de conception (UML, base de donnees, architecture) ; le Chapitre 3 decrit les choix techniques et l'API ; le Chapitre 4 presente les realisations concretes et les tests effectues.

01

1.1 Contexte et Problematique

Le secteur pharmaceutique fait face a des defis croissants : suivi precis des stocks, controle des dates d'expiration, traitement des commandes clients et tenue d'un catalogue a jour. Les methodes manuelles generent des erreurs couteuses, des ruptures de stock non detectees et une experience client degradee.

Nous avons identifie ce besoin reel et propose PharmaCie, une solution web moderne permettant a une pharmacie de gerer centralement ses operations tout en offrant un service de commande en ligne a ses clients.

1.2 Objectifs du Projet

- ▶ Developper une application web full stack complete, responsive et securisee
- ▶ Permettre aux clients de consulter, rechercher et commander des medicaments en ligne
- ▶ Offrir un tableau de bord d'administration pour piloter produits, stocks et commandes
- ▶ Implementer un systeme d'authentification robuste avec gestion des roles (Admin / Client)
- ▶ Garantir l'integrite des donnees grace aux transactions atomiques et au controle de stock
- ▶ Appliquer les meilleures pratiques : MVC, API REST, React Context, Laravel Sanctum

1.3 Parties Prenantes

Partie prenante	Role	Responsabilites
Milad Dlou	Developpeur Frontend	React 19, Vite, Tailwind CSS, React Context, routing
Mouhcine Ait Yahya	Developpeur Backend	Laravel 13, API REST, MySQL, Sanctum, migrations
Pr. Khalid Mzibra	Encadrant pedagogique	Suivi du projet, validation des livrables
ISTA Hay Salam	Commanditaire	Cadre pedagogique, evaluation du projet

1.4 Planning et Organisation

Phase	Activites	Periode
Analyse	Recueil des besoins, cahier des charges, UML	Mai 2026 — S1
Conception	Modelisation BDD, architecture API, maquettes	Mai 2026 — S2
Developpement Backend	Migrations, modeles, controleurs, API, Sanctum	Mai–Juin 2026
Developpement Frontend	Composants React, contexts, routing, API	Mai–Juin 2026
Tests & Integration	Tests fonctionnels, bugs, deploiement	Juin 2026
Documentation	Rapport, presentation, documentation technique	Juin 2026

02

2.1 Acteurs et Cas d'Utilisation

Acteur	Description	Actions disponibles
Visiteur	Utilisateur non authentifié	Consulter le catalogue, s'inscrire, se connecter
Client	Utilisateur connecté (role client)	Catalogue + panier + commandes + profil
Administrateur	Utilisateur connecté (role admin)	Toutes les fonctions + dashboard admin

2.2 Diagramme de Cas d'Utilisation

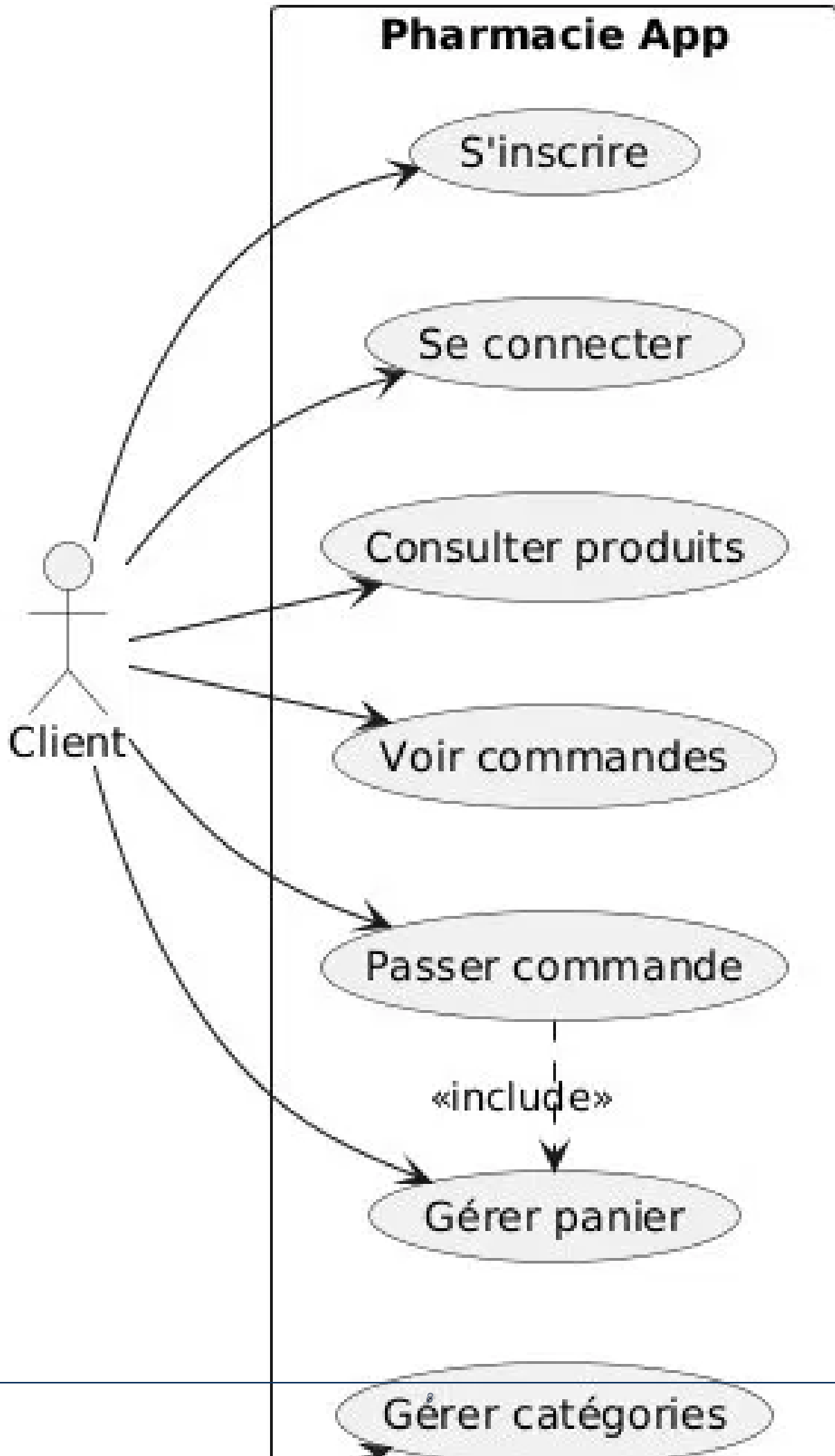


Figure 1 — Diagramme de cas d'utilisation (Use Case)

2.3 Diagramme de Classe

Diagramme de Classes - Application de Gestion de Pharmacie

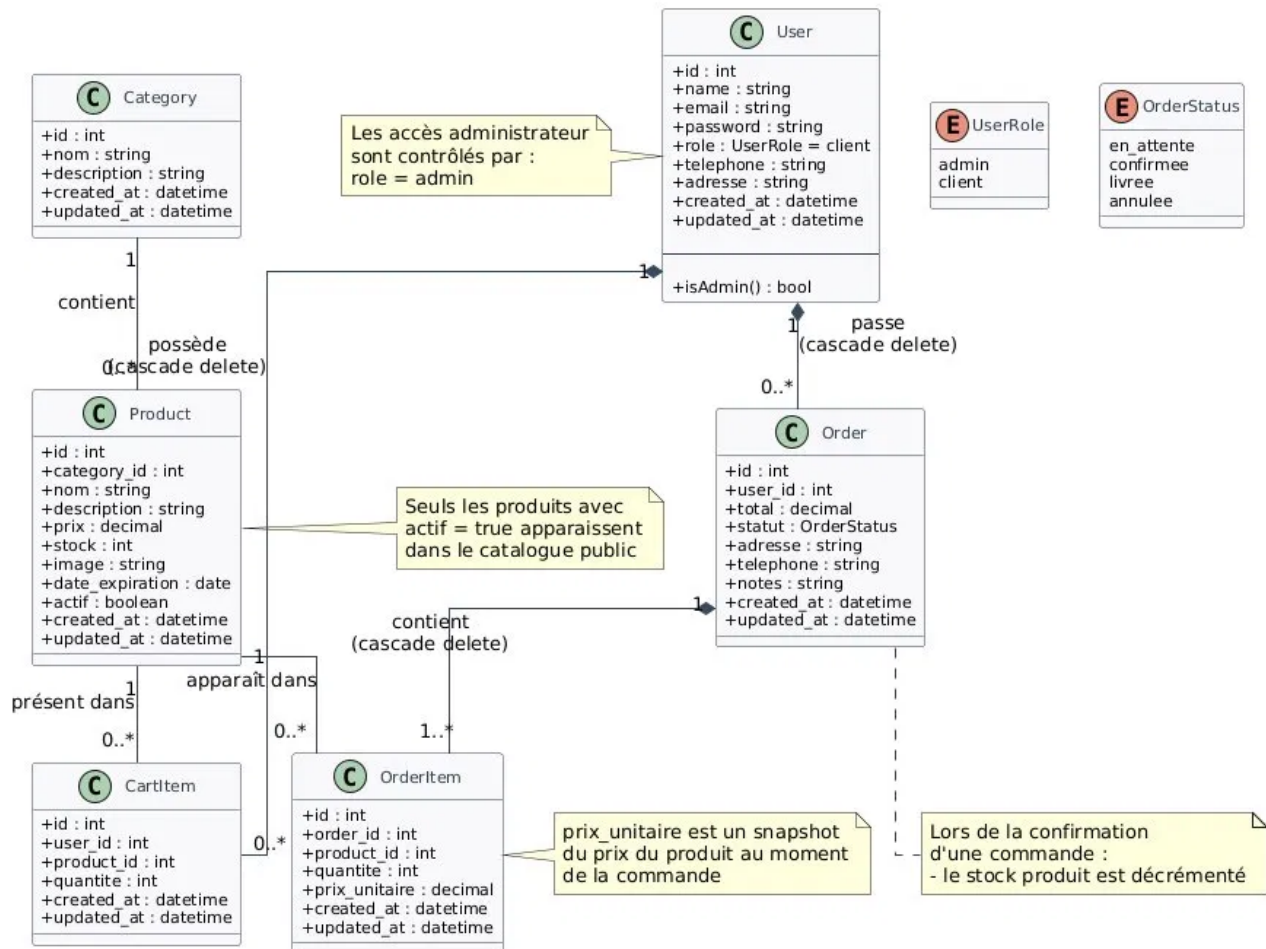


Figure 2 — Diagramme de classe UML

2.4 Modele de la Base de Donnees

La base de donnees MySQL est composee de 6 tables principales reliees par des cles etrangeres :

Table	Colonnes principales	Description
users	id, name, email, password, role, telephone, adresse	Comptes utilisateurs (admin / client)
categories	id, nom, description	Categories pharmaceutiques
products	id, category_id, nom, prix, stock, image, date_expiration, actif	Catalogue medicaments
orders	id, user_id, total, statut, adresse, telephone	Commandes (en_attente / confirmee / livree / annulee)
order_items	id, order_id, product_id, quantite, prix_unitaire	Lignes de commande
cart_items	id, user_id, product_id, quantite	Panier temporaire par utilisateur

Relations Eloquent :

- User hasMany Order, CartItem
- Category hasMany Product | Product belongsTo Category
- Order hasMany OrderItem | OrderItem belongsTo Product

2.5 Architecture du Systeme

L'application adopte une architecture client-serveur decoupee. Frontend et backend sont deux projets independants communiquant via une API REST JSON securisee.

Flux de communication

Navigateur (React SPA) <-> Axios + Bearer Token <-> Laravel API (port 8000) <-> Eloquent ORM <-> MySQL (port 3306)

Couche	Technologie	Responsabilite
Presentation	React 19 + Tailwind CSS	Interface utilisateur reactive et responsive
Communication	Axios + JSON	Appels API avec Bearer Token automatique
Controle	Laravel Controllers	Traitement, validation, logique metier
Modele	Eloquent ORM	Mapping objet-relationnel, transactions
Persistence	MySQL 8.0	Stockage des donnees relationnelles

03

3.1 Stack Technologique — Backend

Technologie	Version	Role
PHP	8.2+	Langage serveur
Laravel	13.x	Framework MVC — routing, controllers, Eloquent, migrations
Laravel Sanctum	4.x	Authentification API par token Bearer
Eloquent ORM	integre	Mapping objet-relationnel, relations
MySQL	8.0	Base de donnees relationnelle
Laravel Storage	integre	Upload et stockage d'images produits (max 2 Mo)

3.2 Stack Technologique — Frontend

Technologie	Version	Role
React.js	19.x	Librairie UI — composants fonctionnels et hooks
Vite	8.x	Bundler avec Hot Module Replacement
Tailwind CSS	4.x	Framework CSS utilitaire — design responsive
React Router	7.x	Routage SPA, routes protegees
Axios	1.x	Client HTTP — appels API avec Bearer Token automatique
React Context API	integre	Gestion etat global : AuthContext, CartContext

3.3 API REST — Endpoints principaux

L'API expose 25 endpoints prefixes par /api, organisees en 3 niveaux d'accès :

Methode	Endpoint	Description	Acces
GET	/products	Catalogue (search, category, pagination)	Public
GET	/products/{id}	Detail produit	Public
GET	/categories	Liste categories	Public
POST	/register	Inscription — retourne { user, token }	Public
POST	/login	Connexion — retourne { user, token }	Public
POST	/logout	Deconnexion — invalide le token	Auth
GET/PUT	/profile	Consulter / modifier le profil	Auth
GET/POST	/cart	Voir / ajouter au panier	Auth
PUT/DEL	/cart/{id}	Modifier quantite / supprimer article	Auth
GET/POST	/orders	Historique / passer une commande	Auth
GET	/admin/orders	Toutes les commandes (paginees)	Admin
PUT	/admin/orders/{id}	Changer le statut d'une commande	Admin
POST/PUT/DEL	/categories/{id}	CRUD categories	Admin
POST/DEL	/products/{id}	Creer / supprimer produit	Admin

3.4 Sécurité et Authentification

Mécanisme — Laravel Sanctum

Après connexion (POST /api/login), le serveur émet un Bearer Token stocké côté client (localStorage). Chaque requête authentifiée l'inclut dans le header Authorization: Bearer <token>. À la déconnexion, le token est révoqué côté serveur.

Mesure de sécurité	Implementation
Hachage mots de passe	bcrypt (12 rounds) via Hash::make()
Contrôle d'accès	AdminMiddleware — HTTP 403 si rôle insuffisant
Validation des entrées	Form Requests Laravel avec règles strictes
Ownership ressources	Vérification panier/commandes appartenant à l'utilisateur
CORS	Restreint aux origines autorisées dans config/cors.php
Transactions DB	Commande encapsulée dans DB::transaction() — atomique
Upload sécurisé	Validation type MIME + taille (PNG/JPG, max 2 Mo)

Contrôleurs Backend

Contrôleur	Responsabilités
AuthController	register, login, logout, getProfile, updateProfile
ProductController	CRUD produits, upload image, filtres catalogue
CategoryController	CRUD catégories avec compteur de produits
CartController	Ajout, modification quantité, suppression, vidage
OrderController	Passage commande (transaction DB), historique, gestion admin

04

4.1 Interfaces Utilisateur — Cote Client

#	Fonctionnalite	Description	Statut
F01	Inscription	Formulaire : nom, email, mot de passe, telephone, adresse	Livre
F02	Connexion	Authentification email/mot de passe — token Sanctum	Livre
F03	Catalogue produits	Grille paginee (20/page) : image, nom, prix, stock	Livre
F04	Recherche temps reel	Filtrage dynamique sur le catalogue	Livre
F05	Filtre par categorie	Selection categorie avec mise a jour instantanee	Livre
F06	Fiche produit	Description, prix, stock, date d'expiration	Livre
F07	Panier	Ajout, modification quantite (min 1 / max stock), suppression	Livre
F08	Validation commande	Adresse + telephone de livraison, deduction stock	Livre
F09	Historique commandes	Liste avec statut, accordéon de detail	Livre
F10	Profil utilisateur	Modifier nom, telephone, adresse, mot de passe	Livre

4.2 Interfaces Utilisateur — Cote Administrateur

#	Fonctionnalite	Description	Statut
A01	Dashboard	Statistiques : produits, categories, commandes en attente	Livre
A02	Gestion produits	CRUD : nom, prix, stock, categorie, image, expiration, actif	Livre
A03	Upload image	PNG/JPG max 2 Mo — stockage Laravel Storage	Livre
A04	Gestion categories	CRUD avec compteur de produits par categorie	Livre
A05	Gestion commandes	Liste paginee avec detail des articles	Livre
A06	Mise a jour statut	en_attente -> confirmee -> livree / annulee	Livre

4.3 Tests Fonctionnels

Les tests ont été réalisés manuellement via le navigateur et Postman pour l'API REST :

Scénario	Résultat attendu	Résultat obtenu
Inscription avec email existant	Erreur 422 — email déjà pris	Conforme
Connexion mauvais mot de passe	Erreur 401 — identifiants invalides	Conforme
Accès admin sans rôle admin	Redirection vers la page d'accueil	Conforme
Ajout panier produit hors stock	Bouton désactivé — stock insuffisant	Conforme
Commande — déduction du stock	Stock diminue atomiquement à la confirmation	Conforme
Commande avec panier vide	Erreur 422 — le panier est vide	Conforme
Upload image > 2 Mo	Erreur validation — fichier trop grand	Conforme
Modification profil (password)	Nouveau mot de passe haché et mis à jour	Conforme
Déconnexion — token révoqué	Token invalide après logout, accès refusé	Conforme

Résultat global — tous les tests sont conformes

L'ensemble des scénarios testés ont produit les résultats attendus. L'application gère correctement les cas limites (stock insuffisant, permissions insuffisantes, données invalides) et maintient la cohérence des données grâce aux transactions atomiques.

Conclusion Generale

Ce projet de fin d'etudes nous a permis de concevoir et de realiser PharmaCie, une application web full stack complete repondant aux besoins reels de gestion d'une pharmacie. Du cahier des charges a la mise en production, en passant par la conception UML, le developpement de l'API REST et la realisation de l'interface React, nous avons mobilise l'ensemble des competences acquises durant notre formation en Developpement Digital Web Full Stack a l'ISTA Hay Salam.

Les principaux apports de ce projet sont :

- ▶ Maitrise de l'architecture client-serveur decouplee (API REST + SPA React)
- ▶ Implementation d'un systeme d'authentification robuste avec Laravel Sanctum et gestion des roles
- ▶ Gestion d'etat global dans React avec le Context API (AuthContext, CartContext)
- ▶ Application des transactions atomiques pour garantir l'integrite des donnees
- ▶ Collaboration efficace en equipe avec repartition claire des responsabilites (Backend / Frontend)

Perspectives d'amelioration pour les versions futures :

- ▶ Integration d'un module de paiement en ligne (Stripe / PayPal)
- ▶ Systeme de notifications push et email pour le suivi des commandes
- ▶ Developpement d'une application mobile (React Native)
- ▶ Module de statistiques avancees avec graphiques et exports PDF/Excel
- ▶ Alertes automatiques pour les stocks faibles et les produits proches d'expiration

Ce projet represente une etape importante dans notre parcours professionnel. Il nous a non seulement permis de consolider nos competences techniques, mais egalement de developper notre sens de la rigueur, de la planification et du travail en equipe — des qualites essentielles pour notre future carriere dans le developpement web.

Bibliographie et Webographie

Documentation Officielle

- ▶ Laravel 13 Documentation — laravel.com/docs
- ▶ Laravel Sanctum — laravel.com/docs/sanctum
- ▶ React 19 Documentation — react.dev
- ▶ Tailwind CSS v4 — tailwindcss.com/docs
- ▶ Vite Build Tool — vitejs.dev
- ▶ React Router v7 — reactrouter.com
- ▶ Axios Documentation — axios-http.com/docs
- ▶ MySQL 8.0 Reference Manual — dev.mysql.com/doc

Ressources Pedagogiques

- ▶ Cours de Developpement Web Full Stack — ISTA Hay Salam, 2025/2026
- ▶ Pr. Khalid Mzibra — Supports de cours Laravel et React

Outils Utilises

Outil	Usage
VS Code	Editeur de code principal
Postman	Tests et documentation de l'API REST
MySQL Workbench	Modelisation et gestion de la base de donnees
GitHub	Versioning et collaboration (depot de code source)
Draw.io / Lucidchart	Conception des diagrammes UML
Railway	Deploiement cloud du backend Laravel

